

Part 1

<https://studio.edgeimpulse.com/public/1010087/live>

```
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.003906
  snake: 0.988281
  updown: 0.000000
  wave: 0.003906
  anomaly score: -0.232772
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.000000
  snake: 0.996094
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.072503
Starting inferencing in 2 seconds...
Sampling...
Predictions (DSP: 109 ms., Classification: 0 ms., Anomaly: 1 ms.):
  idle: 0.996094
  snake: 0.000000
  updown: 0.000000
  wave: 0.000000
  anomaly score: -0.218131
Starting inferencing in 2 seconds...
```

Part 2

Question 1: Model Architecture and Ensemble Flow

Input

Each sample is a sliding window of 100 time steps $\times$ 6 IMU features, flattened to a 600-dimensional input vector.

Three Input Branches

The same raw IMU data is presented in three forms simultaneously:

- Branch A — Raw values
- Branch B — Standard-scaled
- Branch C — Min-max-scaled (range [0, 1])

Encoder Architecture (identical in each branch)

Layer	Details
Input	600 neurons (flattened 100-step × 6-feature window)
Dense 1	64 neurons, ReLU activation
Dense 2 (latent)	32 neurons, Linear activation

The decoder mirrors this path back to 600 outputs and is discarded for deployment.

Classifier Architecture (identical in each branch)

Takes the 32-dimensional latent vector as input:

Layer	Details
Input	32 neurons (latent vector from encoder)
Dense 1	20 neurons, ReLU activation
Output	12 neurons, Softmax activation

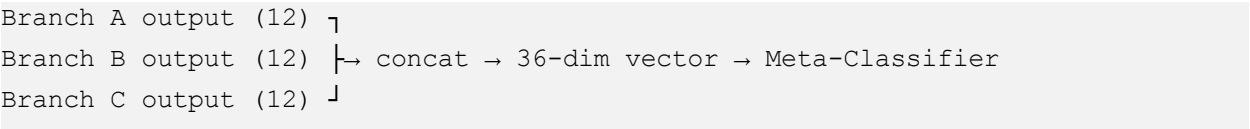
Flattened Branch (Encoder + Classifier Combined)

For deployment, encoder and classifier are merged into a single 4-layer model (confirmed by flattened_raw.summary() output: 41,456 parameters, 161.94 KB baseline):

```
600 → Dense(64, ReLU) → Dense(32, linear) → Dense(20, ReLU) → Dense(12, Softmax)
```

Ensemble Connection

Each of the three branch classifiers outputs a 12-dimensional softmax probability vector. These are concatenated into a 36-dimensional ensemble input:



Stacked Meta-Classifier Architecture

Layer	Details
Input	36 neurons (3 × 12 concatenated softmax outputs)
Dense 1	24 neurons, ReLU activation [meta_dense_24]
Output	12 neurons, Softmax activation [meta_softmax]

Total models in ensemble: 3 branch models + 1 stacked meta-classifier = 4 models

Question 2: Pruning and QAT Methodology

1. Pruning

Pruning is applied using `prune_low_magnitude()` from TF-MOT with a PolynomialDecay sparsity schedule:

Parameter	Value
Initial sparsity	10%
Final sparsity	80%
begin_step / end_step	0 / 500
Pruning epochs	5 epochs with UpdatePruningStep callback
Achieved sparsity (output)	~79.44% overall

2. Why Strip Pruning Wrappers Before QAT

Pruning wrappers add TF-MOT-internal structures (step counters, mask variables, and wrapper layers) around each Dense layer. The `quantize_model()` function expects standard Keras Dense layers and cannot wrap an already-pruning-wrapped layer. Stripping with `strip_pruning()` removes these wrappers and produces a clean Keras model whose weights are already sparse, which `quantize_model()` can then wrap correctly with fake-quantization nodes.

3. QAT Application After Pruning

After stripping, `quantize_mode` inserts fake-quantization nodes at each layer. The model is compiled and trained for 5 more epochs under quantization constraints, so the remaining non-zero weights learn to tolerate the rounding errors introduced by int8 quantization.

4. Why the Mask Enforcement Step Is Needed

Stripping the pruning wrappers allows the Adam optimizer during QAT fine-tuning to update the previously pruned zero weights, which would destroy the sparsity pattern. The `MaskEnforcerCallback` solves this by:

- Extracting binary masks before QAT.
- Multiplying the `QuantizeWrapper` weights by the mask after every training batch and epoch to force pruned positions back to zero, thus preserving the ~80% sparsity.

5. int8 TFLite Conversion and Why Representative Data Is Needed

After QAT, the model is converted with full integer constraints:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
```

QAT already quantizes weights using simulated quantization during training. However, the activation ranges (dynamic range of intermediate layer outputs) must also be quantized for full int8 inference. The converter runs the model on a representative dataset (200 samples drawn uniformly from training data) and records the min/max of each activation, then uses those

ranges to compute int8 scale and zero-point parameters. Without representative data, activation quantization would be uncalibrated and accuracy would degrade severely.

6. How Pruning and int8 Quantization Help on the Arduino

From the notebook outputs:

Model	Baseline TFLite	Compressed int8	Reduction
Each encoder branch	159.87 KB	44.38–44.44 KB	~72%
Stacked meta-classifier	6.46 KB	3.55 KB	~45%

- Pruning (~80% sparsity): ~80% of weights are zero. Combined with quantization, this achieves significant size reduction and reduces multiply-accumulate operations at runtime.
- int8 Quantization: Replaces 32-bit floats with 8-bit integers — a 4× memory reduction — and enables integer arithmetic on the ARM Cortex-M4 (no hardware FPU acceleration for floats), making inference faster and more power-efficient on the constrained Arduino Nano 33 BLE Sense.

Question 3: Arduino Deployment and Real-World Behavior

Serial Monitor Screenshot

```
19:17:57.123 -> Samples collected: 25
19:17:57.681 -> Samples collected: 50
19:17:58.208 -> Samples collected: 75
19:17:58.765 -> Samples collected: 100
19:17:58.765 -> Window collected. Running ensemble inference...
19:17:58.765 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.765 -> Standard-scaled model scores: 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.798 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.3555, 0.0000, 0.5586, 0.0000, 0.0117, 0.0742, 0.0000, 0.0000, 0.0000, 0.0000
19:17:58.798 -> Meta-classifier scores: 0.0195, 0.1680, 0.6016, 0.0625, 0.0000, 0.0039, 0.0000, 0.0391, 0.0664, 0.0273, 0.0000, 0.0078
19:17:58.798 -> -----
19:17:58.798 -> Final prediction: Lying down
19:17:58.798 -> Class index: 2
19:17:58.798 -> Confidence score: 0.6016
19:17:58.798 -> -----
19:17:59.324 -> Samples collected: 25
19:17:59.883 -> Samples collected: 50
19:18:00.413 -> Samples collected: 75
19:18:00.937 -> Samples collected: 100
19:18:00.937 -> Window collected. Running ensemble inference...
19:18:00.968 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:18:00.968 -> Standard-scaled model scores: 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
19:18:00.968 -> Min-max-scaled model scores: 0.0000, 0.0000, 0.3516, 0.0000, 0.5547, 0.0000, 0.0117, 0.0820, 0.0000, 0.0000, 0.0000, 0.0000
19:18:01.001 -> Meta-classifier scores: 0.0195, 0.1641, 0.6094, 0.0625, 0.0000, 0.0039, 0.0000, 0.0391, 0.0664, 0.0273, 0.0000, 0.0078
19:18:01.001 -> -----
19:18:01.001 -> Final prediction: Lying down
```

Output Assessment for a Resting Board

For a board lying flat on a table, the dominant signal is ~1g of gravity acting on the Z-axis (~9.81 m/s²) with near-zero dynamic motion. The closest training classes to this condition are "Standing still," "Lying down," or "Sitting and relaxing" depending on how the board axis aligns with the training data gravity vector. Predictions that remain within these static classes are broadly

acceptable. Unstable oscillation between multiple unrelated classes (e.g., Jogging, Running) would indicate out-of-distribution input.

Observed Issues

- Sensor placement mismatch: The mHealth dataset was collected from body-worn sensors (chest/wrist/ankle). An Arduino flat on a desk produces a completely different signal profile.
- Scaling parameter mismatch: The StandardScaler and MinMaxScaler were fit on mHealth data. If Arduino outputs are in different units or have different axis conventions, all three scaled inputs will be distorted.
- Sensor noise: Even a resting board shows small noise, and with 80% sparse int8 weights, the model's discrimination resolution may be low enough to flip predictions frame-to-frame.

Practical Improvements

- Apply the correct unit conversion (multiply LSM9DS1 g output by 9.81) before feeding into the scaling pipeline.
- Re-collect a small calibration dataset from the actual Arduino sensor and re-fit the StandardScaler and MinMaxScaler with real board data to correct the distribution mismatch.

Question 4: Deployment Behavior on the Arduino

Observed Behavior

The most likely unexpected behavior is that the model repeatedly predicts the same class regardless of how the board is moved, or cycles between a narrow subset of classes (e.g., always predicting "Cycling").

Why This Happens — Possible Causes

1. Training Data vs. Live Data Domain Mismatch

The mHealth dataset was collected from human subjects performing labeled activities with sensors strapped to their bodies. A live Arduino board being manually moved on a desk produces a fundamentally different signal profile in amplitude, frequency content, and axis distribution. The model has never seen a person holding an Arduino and waving it.

2. Sensor Axis Orientation

For a flat Arduino, gravity reads as $\sim +1g$ on the LSM9DS1 Z-axis. In the mHealth dataset, the sensor was body-worn, so gravity distributed across all axes depending on limb angle and body posture. The gravity-relative feature patterns learned during training simply do not map to a bench-held PCB.

3. Quantization Problem

With ~80% sparsity and int8 precision, the model operates with significantly reduced representational capacity. In ambiguous or out-of-distribution regions, the model's confidence can heavily favor whichever class's sparse int8 weight pattern produces the largest dot product.